

Практическая работа №2

Теоретическое введение

Docker Compose — это инструмент для определения и запуска многоконтейнерных Docker-приложений. Он позволяет определить структуру приложения в YAML-формате и управлять этими контейнерами с помощью простых команд командной строки.

Вот несколько ключевых концепций, которые следует учитывать при работе с Docker Compose:

1. Docker Compose файл: Docker Compose использует файл `docker-compose.yml` для определения параметров приложения, таких как сервисы, сети, тома и другие опции. В этом файле вы определяете сервисы, их зависимости, сети и объемы данных.

2. Сервисы: Сервисы представляют собой отдельные контейнеры, которые составляют ваше приложение. Каждый сервис определяется в файле `docker-compose.yml` и может включать конфигурацию контейнера, такую как образ Docker, команды запуска, переменные среды, порты и другие параметры.

3. Сети и тома: Docker Compose позволяет определять пользовательские сети и тома данных для ваших сервисов. Это позволяет контролировать способ взаимодействия между контейнерами и обеспечить сохранение данных между запусками контейнеров.

4. Управление жизненным циклом: Docker Compose упрощает управление жизненным циклом контейнеров вашего приложения. Вы можете легко создавать, запускать, останавливать и удалять сервисы с помощью простых команд, таких как `docker-compose up` и `docker-compose down`.

5. Масштабирование: Docker Compose также позволяет масштабировать сервисы горизонтально, то есть запускать несколько экземпляров сервисов, чтобы обеспечить балансировку нагрузки и повысить производительность вашего приложения.

В целом, Docker Compose обеспечивает простой и эффективный способ управления многоконтейнерными приложениями с помощью интуитивно понятного YAML-синтаксиса и набора команд командной строки. Он позволяет разработчикам и DevOps-специалистам быстро и легко разворачивать и управлять приложениями в среде Docker.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>

Задание

Практическое задание: Развертывание многоконтейнерного приложения с использованием Docker Compose

Цель: освоить основы Docker Compose и научиться развертывать, и управлять многоконтейнерными приложениями с помощью файла ``docker-compose.yml``.

Задачи:

1. Установка Docker и Docker Compose:

- Убедитесь, что Docker и Docker Compose установлены на вашей системе. Если нет, выполните установку, следуя инструкциям в официальной документации Docker.

2. Создание многоконтейнерного приложения:

- Создайте папку для вашего проекта.
- В этой папке создайте файл ``docker-compose.yml``.
- Определите в файле ``docker-compose.yml`` сервисы, необходимые для вашего приложения. Например, это может быть веб-сервер, база данных и кэш Redis.

3. Настройка сервисов:

- Для каждого сервиса определите настройки, такие как образ контейнера, порты, переменные среды и т. д.
- Убедитесь, что все сервисы взаимодействуют друг с другом правильно.

4. Запуск приложения:

- Запустите многоконтейнерное приложение с помощью команды ``docker-compose up``.
- Убедитесь, что все контейнеры успешно поднялись, и приложение работает корректно.

5. Тестирование функциональности:

- Протестируйте функциональность вашего приложения, убедитесь, что оно работает так, как задумано.

6. Остановка и удаление контейнеров:

- После завершения работы с приложением остановите и удалите все контейнеры с помощью команды ``docker-compose down``.

Установка

Перед началом работы с Docker-compose необходимо удостовериться в том что он установлен в вашей системе. Обычно он устанавливается сразу вместе с Docker, но проверить его наличие всегда можно с помощью команды:

```
docker-compose version
```

Если в результате вы видите актуальную версию Docker-compose, значит он установлен. В ином случае вы можете установить его, следуя официальным инструкциям на сайте Docker: <https://docs.docker.com/compose/install/>.

Создание многоконтейнерного приложения

Для работы с Docker-compose нам необходимо создать файл `docker-compose.yml` в рабочей директории. Именно этот файл позволяет одновременно запустить несколько контейнеров одновременно и настроить их конфигурацию.

Файл `docker-compose.yml` используется для определения сервисов, сетей и объемов данных, которые должны быть созданы и запущены с помощью Docker Compose.

Основные элементы файла Docker-compose:

`version`: определяет версию формата файла `docker-compose.yml`.

`services`: секция, в которой определяются все сервисы, которые вы хотите создать и запустить.

`service1`, `service2`: названия сервисов, каждый из которых может иметь несколько конфигурационных параметров.

`image`: используемый образ docker или его название и тег. Используется для создания сервиса на основе существующего образа.

build: опция, позволяющая указать dockerfile и контекст сборки для создания образа из исходных кодов.

ports: определяет проброс портов между хостом и контейнером.

volumes: позволяет монтировать тома данных в контейнере.

environment: устанавливает переменные среды для контейнера.

networks: подключает сервис к существующей сети docker.

volumes: определяет тома данных, которые могут использоваться сервисами.

networks: определяет сети, которые могут использоваться сервисами.

Пример содержимого файла Docker-compose:

```
version: '3.8'
```

```
services:
```

```
  web:
```

```
    build: .
```

```
    ports:
```

```
      - "5000:5000"
```

```
    environment:
```

```
      - FLASK_ENV=development
```

```
    volumes:
```

```
      - ./app
```

```
    depends_on:
```

```
      - db
```

```
      - redis
```

```
  db:
```

```
    image: postgres:latest
```

```
    environment:
```

```
      - POSTGRES_USER=app_user
```

```
      - POSTGRES_PASSWORD=securepassword
```

```
      - POSTGRES_DB=mydatabase
```

```
    ports:
```

```
      - "5432:5432"
```

```
    volumes:
```

```
      - pgdata:/var/lib/postgresql/data
```

```
redis:
  image: redis:latest
  ports:
    - "6379:6379"
```

```
volumes:
  pgdata:
```

В данном файле происходит настройка подключаемых сервисов и портов.

В нашем случае это подключение Postgres и Redis.

Также необходимо создать Dockerfile для работы с основным приложением.

Для примера представлен Dockerfile для работы с Postgres и Redis на языке Python с использованием Flask.

Пример содержимого Dockerfile:

Устанавливаем базовый образ Python

```
FROM python:3.9
```

```
# Устанавливаем переменную окружения для Python, чтобы
не выводить логи
```

```
ENV PYTHONUNBUFFERED 1
```

```
# Устанавливаем директорию рабочей области внутри
контейнера
```

```
WORKDIR /app
```

```
# Копируем файлы приложения и requirements.txt в
контейнер
```

```
COPY requirements.txt /app/
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Копируем остальные файлы приложения в контейнер
```

```
COPY . /app/
```

```
# Команда для запуска приложения Flask
```

```
CMD ["python", "app.py"]
```

Пример запускаемой программы (app.py):

```
import redis
import psycopg2

# Подключение к Redis
redis_host = 'redis'
redis_port = 6379
redis_db = 0
redis_client = redis.StrictRedis(host=redis_host,
port=redis_port, db=redis_db)

# Подключение к PostgreSQL
pg_host = 'db'
pg_port = 5432
pg_db = 'mydatabase'
pg_user = 'app_user'
pg_password = 'securepassword'
conn = psycopg2.connect(host=pg_host, port=pg_port,
database=pg_db, user=pg_user, password=pg_password)
cursor = conn.cursor()

# Сохранение данных в Redis
redis_key = 'example_key'
redis_value = 'example_value'
redis_client.set(redis_key, redis_value)

# Получение данных из Redis
value_from_redis = redis_client.get(redis_key)
print(f'Data from Redis: {value_from_redis.decode()}')

# Создание таблицы в PostgreSQL
create_table_query = '''
CREATE TABLE IF NOT EXISTS example_table (
    id SERIAL PRIMARY KEY,
    data TEXT
)
'''
cursor.execute(create_table_query)
```

```

conn.commit()

# Вставка данных в PostgreSQL
pg_value = 'data_for_postgres'
insert_query = 'INSERT INTO example_table (data) VALUES (%s) '
cursor.execute(insert_query, (pg_value,))
conn.commit()

# Получение данных из PostgreSQL
select_query = 'SELECT * FROM example_table'
cursor.execute(select_query)
records = cursor.fetchall()
print('Data from PostgreSQL:')
for record in records:
    print(record)

```

Для нашего примера также необходим файл requirements.txt для правильной работы Flask.

Пример содержимого файла requirements.txt:

```

Flask==2.0.1
psycopg2==2.9.2
redis==3.5.3

```

Запуск контейнеров

После того, как все настроено, нам требуется запустить созданные контейнеры. Сделать это можно с помощью команды:

```
sudo docker-compose up -d -build
```

Для проверки успешности запуска контейнеров и работы программы можно использовать:

```
sudo docker ps
```

для просмотра запущенных контейнеров и:

```
docker logs <имя_контейнера_с_программой>
```

для просмотра вывода программы в контейнере.

Остановка и удаление контейнеров

Для остановки и удаления контейнеров используйте команду:

```
docker-compose down
```

Вопросы к практической работе

1. Что такое Docker Compose и для чего он используется?
2. Какой синтаксис используется для определения сервисов в файле `docker-compose.yml`?
3. Какие основные разделы включает файл `docker-compose.yml`?
4. Как создать сервис на основе существующего образа Docker с помощью Docker Compose?
5. Какой ключевой параметр используется для сборки образа из исходных кодов в Docker Compose?
6. Какие опции позволяют настроить проброс портов между хостом и контейнером в Docker Compose?
7. Как создать и использовать тома данных с помощью Docker Compose?
8. Как устанавливать переменные среды для контейнеров в Docker Compose?
9. Как подключить сервис к существующей сети Docker в Docker Compose?
10. Как запустить все сервисы, описанные в файле `docker-compose.yml`, с помощью команды Docker Compose?

Критерии оценки

- Установлен Docker-compose
- Создан файл Docker-compose
- Развернуто контейнеры с собственным приложением и сервисами с помощью Docker-compose:
- Проверена работоспособность программы и контейнеров
- Остановлены и удалены контейнеры с помощью Docker-compose
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе